

AI技术调研与场景分析

一、报告说明

报告以第一次作业完成的“辞典编撰辅助平台开发”软件过程定义为基础，结合近年来AI for Software Engineering (AI4SE) 发展趋势，分析人工智能技术在需求分析、系统设计、编码实现、测试验证、运维监控等软件生命周期环节中的典型应用，并进一步识别本项目中最适合优先引入AI能力的关键过程。

本项目的对象是“基于训诂学的汉字释义知识图谱构建与辞典编撰辅助平台开发”。系统面向辞典编撰专家和古汉语研究者，通过Neo4j、MySQL、React、FastAPI、Graph-RAG、大模型和多智能体等技术，支撑“审、比、勘、改”的训诂思维链。由于项目同时具有软件工程、知识工程、自然语言处理和领域专家协同等特征，AI技术的引入不能只停留在“代码补全”层面，而应与需求澄清、知识建模、智能体编排、证据链验证和专家闭环相结合。

二、AI4SE行业趋势调研

2024年以来，生成式AI、大模型和智能体技术开始系统性进入软件工程领域，AI4SE从单点工具试用逐渐转向全生命周期过程重构。中国信息通信研究院《AI4SE行业现状调查报告（2026年）》显示，受访企业对软件研发智能化转型的重视程度明显提升，2025年“非常重视”的比例达到46.09%，相较2024年提高约26个百分点；处于L3核心智能化阶段的企业占比从17.62%提升至29.75%；41.08%的企业在软件研发中应用大模型已超过一年[1]。Google Cloud DORA 2024报告也指出，AI正在改变软件开发日常流程，但团队仍需稳定组织优先级、持续实验和面向用户价值进行改进[8]；DORA 2025进一步将AI定位为“放大器”，强调AI收益不只来自工具本身，还取决于组织过程、度量体系和工程治理能力[9]。这些研究共同说明，AI辅助软件工程已不再只是实验性尝试，而正在成为企业研发流程中的常态化能力。

从技术落地方式看，企业普遍倾向于低成本、可集成、可治理的应用路径。中国信通院报告指出，私有化部署中通过RAG嵌入私有知识库的方式占比最高，达到40.35%，全栈自研或微调方式占比不足20%[1]；《智能化软件开发落地实践指南（2024）》也强调，智能化软件开发需要围绕研发流程、工具平台、知识资产和组织协同形成落地闭环[2]。Stack Overflow 2024与2025开发者调查均显示，AI工具已经成为开发者工作流程中的重要工具，但开发者对AI输出准确性和可信度仍保持谨慎态度[10][11]。因此，辞典编撰辅助平台并不适合完全依赖通用大模型直接生成结论，而需要将权威辞书、书证、专家规则和图谱结构纳入检索增强上下文，使模型输出受到证据约束，降低幻觉风险。

从效率提升看，AI对软件工程各环节均产生了较明显影响，但不同研究对“提效”的结论并不完全一致，需要交叉理解。中国信通院报告显示，2025年开发环节效率提升达到32.63%，运维环节达到36.36%；智能测试工具生成文本测试用例的平均采纳率达到43.26%，测试效率提升30%以上的企业占比达到66.23%[1]。JetBrains 2024开发者生态报告显示，ChatGPT、GitHub Copilot等工具已进入开发者日常工具箱[12]；一项基于Google工程师的随机对照实验估计，AI功能可使复杂企业级任务耗时缩短约21%[6]。但METR在2025年针对资深开源开发者的实验发现，在成熟代码库和高质量约束下，AI工具反而可能增加完成时间[7]；Sonar 2026报告也强调，AI生成代码仍需要开发者进行审查和质量把关[13]。这说明AI4SE的价值不是简单“替代人工”，而是需要配套代码审查、测试验证、质量门禁和人工复核机制。

从工具形态看，AI正在从“代码补全助手”发展为更强的研发协作者。2026年的生成式AI软件开发综述指出，GenAI在设计、实现、测试和文档任务中影响较高，并推动开发者工作重心转向规约表达、架构推理和结果监督[4]；关于2030年开发者工作的研究也认为，AI辅助编程将使开发者从手工编码者逐步转向AI驱动开发生态的编排者[5]。GitHub Copilot等工具文档表明，当前AI编码工具已经覆盖代码生成、解释、测试、审查和开发问答等多类任务[14]，而中国关于代码大模型、智能测试能力等标准化文件则反映出行业正尝试把AI工具能力转化为可评估、可验收的工程能力[3][15]。

总体来看，近年来的AI4SE呈现出四个明显趋势：第一，AI工具从个人效率工具扩展为覆盖需求、设计、编码、测试、运维的过程能力[1][2][8][9]；第二，软件工程产物正在从“人类可读”转向“AI可消费”，需求说明、接口规约、测试用例和运行日志都需要被结构化，以便后续Agent读取和执行[4][5]；第三，人机协作边界从“AI给建议、人写代码”扩展为“人定义目标和约束，AI执行部分工程任务，人负责验证和决策”[5][9][14]；第四，AI落地的关键不只是工具引入，还包括质量门禁、数据安全、结果验证和持续度量[3][11][13][15]。

三、软件生命周期中的AI应用场景

生命周期阶段	AI典型应用	适用技术或工具形态	与本项目的关系
需求分析	访谈纪要整理、需求条目抽取、需求文档生成、需求一致性检查、验收标准生成	大语言模型、需求RAG、语义聚类、规约生成	可将专家访谈、辞书体例、作业文档整理为需求规格、术语表和验收准则[2][4]
系统设计	架构方案生成、UML/流程图生成、接口设计、数据库表结构建议、设计模式推荐	架构Agent、模型驱动设计、Mermaid/UML生成、知识图谱建模辅助	可辅助设计Graph-RAG链路、Neo4j图谱结构、FastAPI接口和“审比勘改”流程[4][5]

生命周期阶段	AI典型应用	适用技术或工具形态	与本项目的关系
编码实现	代码生成、智能补全、单元函数实现、代码重构、代码审查、注释与文档生成	GitHub Copilot、通义灵码、CodeGeeX、AI IDE、代码大模型	可提高React页面、FastAPI接口、数据库查询、提示词模板管理等开发效率[6][12][14]
测试验证	测试用例生成、测试数据生成、接口测试脚本生成、UI自动化脚本、自愈测试、缺陷预测	智能测试平台、LLM+Pytest、LLM+Playwright、Testin XAgent等	可围绕字头查询、义项对齐、书证溯源、Agent输出格式生成测试集[1][3][10]
运维监控	日志摘要、异常检测、根因分析、告警归并、容量预测、模型质量监控	AIOps、大模型运维问答、日志RAG、异常检测模型	可监控Graph-RAG命中率、模型响应延迟、幻觉率、专家采纳率和接口稳定性[1][8][9]

从表中可以看出，AI可以覆盖软件生命周期各阶段，但不同阶段的价值、风险和实施成本并不相同。对于辞典编撰辅助平台而言，需求和测试阶段尤其重要，因为领域知识复杂、正确性标准依赖专家判断，若需求规则和验证标准不清晰，后续模型推理与系统开发都会受到影响。编码阶段同样适合优先引入AI，因为本项目需要集成前端、后端、图数据库、大模型服务和智能体流程，小团队开发任务较重，AI辅助编码可以直接降低实现成本。

四、本项目AI应用场景匹配分析

结合前期软件过程定义，本项目原过程包括五个核心阶段：领域需求与数据准备、核心算法与引擎研发、系统架构与增量开发、人机协同与闭环验证、规范定稿与部署发布。为了选择合适的AI改进环节，本文从“效率提升空间、数据可获得性、实现成本、质量风险、与项目核心目标的匹配度”五个维度进行分析。由于本项目兼具专业文献研读、知识图谱构建、大模型推理、前后端工程实现和专家闭环验证等特点，AI可介入的场景并不局限于代码生成，而是覆盖从需求理解到交付维护的多个环节。

候选场景	对应原过程	AI介入方式	优先级	选择理由
场景1：AI辅助需求分析	领域需求与数据准备过程	使用自然语言处理和大语言模型研读课程文档、专业文献、专家访谈纪要和辞书体例，提取需求要点、生成需求文档初稿、检查需求一致性	高	本项目需要理解大量专业性需求表达具有训诂学背景，AI可显著降低前期研读和整理成本
场景2：AI辅助专业文献理解与知识抽取	领域需求与数据准备过程	对训诂学论文、辞书说明、书证材料进行摘要、实体识别、术语归一和证据片段标注	中高	有助于沉淀领域知识库，但抽取结果必须由专家复核，适合作为需求分析的支撑材料
场景3：AI辅助语料清洗与三元组抽取	领域需求与数据准备过程	从多源辞书文本中辅助抽取“字头-义项-书证”三元组，发现格式异常、缺失字段和重复记录	中高	直接影响知识图谱质量，但数据清洗规则复杂，需和人工校验流程配套使用
场景4：AI辅助Graph-RAG检索策略优化	核心算法与引擎研发过程	辅助设计宏观训诂规则检索、微观书证检索、检索结果重排和证据上下文组装策略	中	与项目核心技术高度相关，但算法效果评估难度较高，适合在基础流程稳定后迭代优化
场景5：AI辅助多智能体角色与提示词设计	核心算法与引擎研发过程	生成保守派、革新派、裁判Agent的角色说明、提示词模板、工具调用边界和输出格式约束	中	能提升Agent搭建效率，但容易产生看似合理但缺乏专业性的输出，需要专家强复核
场景6：AI驱动的代码生成与审查	系统架构与增量开发过程	在React、FastAPI、数据库访问、接口封装、日志记录等模块中使用AI IDE生成代码并进行审查	高	小团队开发压力大，AI辅助能直接提升工程实现效率
场景7：AI辅助测试用例与质量分析	人机协同与闭环验证过程	自动生成测试用例、智能分析测试覆盖率、辅助预测潜在缺陷，并生成接口测试、UI测试和专家验收清单	高	测试设计工作量大，且本项目存在复杂推理链条，AI可显著提升测试效率和覆盖率
场景8：AI辅助运维监控与模型质量分析	规范定稿与部署发布过程	使用日志RAG和异常检测分析接口错误、响应延迟、Graph-RAG命中率、模型幻觉率和专家采纳率	中	后期价值较高，但依赖系统上线后的真实数据，适合作为持续改进阶段的辅助工具

从匹配结果看，场景2、场景3、场景4和场景5都与项目特色密切相关，特别是专业文献理解、语料清洗、Graph-RAG策略和多智能体提示词设计，能够提升系统的领域智能化水平。但这些场景对数据质量和专家判断依赖较强，短期内难以仅凭工具验证其效果。场景8则更偏向系统上线后的持续监控，需要积累

真实运行日志和专家反馈数据。因此，本阶段不宜把所有AI能力一次性纳入核心改进范围，而应优先选择最容易落地、最容易验证、对软件过程影响最直接的环节。

综合上述分析，本阶段建议最后将“需求、编码、测试”确定为3个核心改进场景。需求阶段决定项目能否准确理解辞典编撰专家的真实工作方式和专业文档要求；编码阶段直接影响小团队的系统交付速度；测试阶段决定AI输出、接口功能和专家闭环结果是否能够被信任和复核。这三个环节既覆盖了前期定义的软件过程主线，又具备较强的可落地性和可验证性。

1. 核心场景一：AI辅助需求分析

本项目的需求来源并不是普通业务系统中的简单功能列表，而是来自训诂学专家的工作流、辞书修订规范、多源古汉语语料和“审比勘改”的领域方法。团队在需求分析阶段需要研读大量专业性文档，包括辞书编撰背景材料、训诂学规则说明、知识图谱构建方案和专家反馈记录。传统人工方式需要反复阅读材料、整理概念和编写需求文档，容易出现术语不统一、需求遗漏、验收标准不清晰和隐性规则表达不足等问题。

引入AI后，可以利用自然语言处理技术和大语言模型辅助完成需求理解工作。具体来说，对于不熟悉古汉语专业的计算机工作者，AI可以帮助阅读晦涩难懂、极度专业化的文献，古籍和训诂学专著，快速对其需求描述，并生成符合项目要求的需求文档。具体应用场景包括：古籍文本智能解析、专业术语自动标注、需求文档自动生成等等。

这种AI辅助的需求工程方法不仅大幅提升了跨专业领域项目的需求获取效率，还显著降低了因专业壁垒导致的沟通成本和需求错误风险，为古籍数字化、传统文化传承等领域的软件开发提供了强有力的技术支撑。

然而，在实际应用中仍需注意AI模型的局限性。由于古汉语语料的稀缺性和专业性，通用大语言模型在处理高度专业化古籍内容时可能存在理解偏差。因此，最佳实践是采用"AI辅助+专家验证"的混合模式，即AI负责初步的需求提取和文档生成，而古汉语专家负责关键内容的审核和修正。这种人机协作的方式既能发挥AI的高效处理能力，又能确保专业内容的准确性，从而实现技术与人文的有机结合，推动传统文化数字化项目的高质量发展。

在具体实施过程中，建议采用渐进式引入策略：首先，将训诂学专著、辞书体例和专家访谈记录等原始材料进行结构化预处理，构建包含《说文解字》、《康熙字典》等权威辞书的私有知识库，通过RAG技术为大语言模型提供可靠的上下文支持；其次，开发专门针对古汉语特点的提示词模板，引导模型准确理解训诂学术语和文献结构，确保AI输出基于权威资料而非通用语料；再次，通过多轮迭代的方式，让AI生成初步需求文档后，由古汉语专家进行逐条审核和修正，并将专家反馈作为新的训练数据持续优化模型表现，建立专家反馈闭环机制；最后，建立需求追溯机制，确保每个需求条目都能关联到具体的文献出处或专家意见，形成完整的需求证据链。

通过这种系统化的方法，AI辅助需求分析不仅能够显著提升跨专业团队的沟通效率，降低因专业壁垒导致的理解偏差，还能在保证学术严谨性的前提下，为辞典编撰辅助平台的后续开发奠定坚实的需求基础，实现技术与人文的有机结合，推动传统文化数字化项目的高质量发展。

2. 核心场景二：AI驱动的代码生成与审查

辞典编撰辅助平台需要同时开发前端页面、后端接口、数据库查询、模型调用服务和智能体编排逻辑。对于四人左右的小团队而言，重复性的工程实现、接口样板代码、数据结构转换和文档注释会占用大量时间。2026年AI4SE行业报告显示，2025年代码生成采纳率平均值达到42.61%，代码入库采纳率达到40.67%^[1]；Stack Overflow 2024调查中，正在使用AI工具的开发者的主要将其用于编写代码、搜索答案、调试和文档编写^[10]；JetBrains 2024报告也显示，AI编程工具已被大量开发者尝试和日常使用^[12]。因此，AI辅助编码已具有较强工程实用性，但仍需要结合Sonar等报告提示的验证风险，在提交前进行人工审查和自动化测试^[13]。

在本项目中，AI辅助编码可重点用于三个方面：第一，生成React组件、表格展示、检索结果卡片、专家反馈表单等前端界面代码；第二，生成FastAPI接口、请求参数校验、Neo4j/MySQL查询封装和错误处理逻辑；第三，在提交前对代码进行智能审查，检查空值处理、接口异常、敏感数据暴露、重复逻辑和潜在性能问题。该场景的关键质量目标是“AI生成代码必须经过人工审查、单元测试和集成测试后才能合并”，避免将模型生成的错误实现直接纳入主干。

3. 核心场景三：AI辅助测试用例

本项目测试难点不只在接口是否可用，还在于“模型输出是否有证据、推理链是否可解释、专家是否认可”。传统测试方法可以验证API状态码、数据库读写和页面交互，但难以覆盖义项比对、书证溯源、多智能体辩论等复杂场景。AI辅助测试可以根据需求规约、接口说明和典型字头数据自动生成测试用例，覆盖正常路径、边界条件、异常输入和证据缺失等情况，尤其适用于复杂系统中的测试设计和质量分析。

具体而言，AI可辅助生成以下测试资产：接口测试用例，如“输入某字头后返回义项、书证和来源字段”；Graph-RAG检索测试用例，如“当检索结果缺少权威书证时，系统不得生成确定性修订建议”；Agent输出格式测试用例，如“保守派、革新派、裁判Agent必须分别输出立场、证据、结论和置信度”；专家验收清单，如“专家能够追溯每条修订建议的来源材料”。在测试执行后，AI还可以辅助分析测试覆盖率，提示哪些需求、接口、异常路径和专家验收点尚未覆盖；也可以结合历史缺陷、接口复杂度和模块变更记录进行预测性缺陷检测，提示高风险模块优先测试。该场景的价值在于把抽象的学术质量要求转化为可执行、可复核的测试条目，从而显著提升测试效率和质量。

五、价值与可行性说明

从价值角度看，AI技术可以帮助本项目解决三类核心问题。第一，降低领域知识整理成本。需求分析和数据准备阶段可以借助AI快速形成结构化草稿，使团队将更多精力放在专家审核和规则修正上。第二，提高工程实现效率。前后端、接口、数据库和测试脚本中存在大量模式化工作，适合由AI辅助完成初稿，

再由开发者把关。第三，增强验证覆盖度。AI能够从需求和接口文档中反向生成测试点，帮助团队发现遗漏场景，提升专家闭环验证的完整性。

从可行性角度看，本项目具备较好的AI落地基础。一方面，项目本身已经采用Graph-RAG、大模型和多智能体技术，团队对AI工具的使用场景较熟悉；另一方面，前期软件过程定义已经明确了数据准备、引擎研发、系统集成、闭环验证和部署维护等阶段，便于将AI节点嵌入现有流程。此外，作业要求的原型验证可优先选择“AI生成测试用例”或“AI生成需求规格草稿”进行最小可行验证，不需要一次性改造完整系统。

需要注意的是，AI在本项目中不能替代专家判断。辞典编撰具有严肃的学术属性，模型可能产生幻觉、误引书证、混淆义项或过度推断。因此，AI输出应被定位为“草稿、建议和辅助分析”，正式结果必须经过人工审核、证据溯源和专家确认。后续过程改进方案中应进一步定义AI输出物的质量标准，例如引用必须可追溯、结论必须附证据、生成代码必须通过测试、测试用例必须经过人工抽查等。

综上，本文建议本项目在第一轮AI过程改进中优先选择三个场景：AI辅助需求分析、AI驱动的代码生成与审查、AI辅助测试用例。这三个场景既符合AI4SE行业趋势，也与辞典编撰辅助平台的真实过程痛点高度匹配，能够为后续“软件过程改进方案设计”和“AI技术应用验证报告”提供明确基础。

参考文献

[1] 中国信息通信研究院人工智能研究所. AI4SE行业现状调查报告（2026年）[R/OL]. 北京: 中国信息通信研究院, 2026[2026-05-30]. <https://www.caict.ac.cn/kxyj/qwfb/ztbg/202604/P020260402599647343221.pdf>.

[2] 中国信息通信研究院, 华为云. 智能化软件开发落地实践指南（2024）[R/OL]. 北京: 中国信息通信研究院, 2024[2026-05-30]. <https://blockchain.caict.ac.cn/kxyj/qwfb/ztbg/202409/P020240919585073158926.pdf>.

[3] 中国人工智能产业发展联盟. 智能化软件工程技术和应用要求 第3部分: 智能测试能力[S/OL]. AIIA/PG 0138-2024, 2024[2026-05-30]. https://www.sohu.com/a/843331181_121124361.

[4] Gurgul V, Gubela R, Lessmann S. The State of Generative AI in Software Development: Insights from Literature and a Developer Survey[J/OL]. arXiv:2603.16975, 2026[2026-05-30]. <https://arxiv.org/abs/2603.16975>.

[5] Qiu K, Puccinelli N, Ciniselli M, Di Grazia L. From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030[J/OL]. arXiv:2405.12731, 2024[2026-05-30]. <https://arxiv.org/abs/2405.12731>.

[6] Paradis E, Grey K, Madison Q, et al. How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial[J/OL]. arXiv:2410.12944, 2024[2026-05-30]. <https://arxiv.org/abs/2410.12944>.

[7] Becker J, Rush N, Barnes E, Rein D. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity[J/OL]. arXiv:2507.09089, 2025[2026-05-30]. <https://arxiv.org/abs/2507.09089>.

[8] Google Cloud DORA. Accelerate State of DevOps Report 2024[R/OL]. 2024[2026-05-30]. <https://dora.dev/research/2024/dora-report/>.

[9] Google Cloud DORA. State of AI-Assisted Software Development 2025[R/OL]. 2025[2026-05-30]. <https://dora.dev/research/2025/dora-report/>.

[10] Stack Overflow. 2024 Developer Survey: Artificial Intelligence[EB/OL]. 2024[2026-05-30]. <https://survey.stackoverflow.co/2024/ai>.

[11] Stack Overflow. 2025 Developer Survey: Artificial Intelligence[EB/OL]. 2025[2026-05-30]. <https://survey.stackoverflow.co/2025/ai>.

[12] JetBrains. The State of Developer Ecosystem 2024[R/OL]. 2024[2026-05-30]. <https://www.jetbrains.com/lp/devecosystem-2024/>.

[13] SonarSource. State of Code Developer Survey Report: The Current Reality of AI Coding[R/OL]. 2026[2026-05-30]. <https://www.sonarsource.com/blog/state-of-code-developer-survey-report-the-current-reality-of-ai-coding>.

[14] GitHub. GitHub Copilot Documentation[EB/OL]. 2026[2026-05-30]. <https://docs.github.com/en/copilot>.

[15] 中国人工智能产业发展联盟. 智能化软件工程技术和应用要求 第1部分: 代码大模型[S/OL]. 2024[2026-05-30]. <https://m.c114.com.cn/w5339-1254140.html>.